

Paragliding Intelligence Platform

Volume 09 - Security Architecture v2 Technical

Security, privacy, compliance, and safety-control architecture for a paragliding-first meteo, tracking, AI briefing, and subscription platform.

Field	Value
Document status	Technical v2 - developer-ready baseline
Primary audience	Founders, CTO, backend/mobile engineers, DevOps, security lead, legal/privacy reviewer
Scope	Authentication, authorization, API security, mobile security, location privacy, AI security, infrastructure security, compliance, incident response
Replaces	Placeholder security notes in earlier volumes

Table of Contents

1. 1. Security Executive Summary
2. 2. Scope, Assumptions, and Trust Boundaries
3. 3. Security Principles and Control Objectives
4. 4. Asset Inventory and Data Classification
5. 5. Threat Model and Abuse Cases
6. 6. Identity Architecture
7. 7. Authentication Flows
8. 8. Authorization: RBAC, ABAC, and Row-Level Security
9. 9. API Security Architecture
10. 10. Mobile Application Security
11. 11. Location Privacy and Live Tracking Controls
12. 12. Data Protection, Encryption, and Key Management
13. 13. GDPR and Privacy Operations
14. 14. AI and LLM Security
15. 15. ML, Weather Data, and Model Supply Chain Security
16. 16. Payment and Subscription Security
17. 17. Cloud, Kubernetes, and Network Security
18. 18. Secrets Management
19. 19. Logging, Audit, and Security Monitoring
20. 20. Incident Response and Business Continuity
21. 21. Security Testing Program
22. 22. Compliance Mapping
23. 23. MVP Security Slice
24. 24. Security Backlog and Acceptance Criteria
25. 25. References

1. Security Executive Summary

This platform is not a normal weather app. It processes precise location, live flight tracking, pilot safety decisions, payments, AI briefings, club/school administration, event operations, and potentially rescue workflows. Security must therefore be designed as a core product capability, not an afterthought.

The architecture follows defense-in-depth: identity hardening, least privilege, tenant isolation, object-level authorization, encryption, location privacy, safe AI grounding, audited administrative actions, monitored infrastructure, and explicit safety fallbacks. The goal is to prevent data exposure, prevent unauthorized tracking, protect payment and entitlement flows, and avoid unsafe AI or weather recommendations caused by tampering or stale data.

Security controls are mapped to established references including OWASP ASVS, OWASP API Security Top 10, OWASP MASVS, OWASP LLM Top 10, NIST CSF 2.0, OAuth 2.0 Security Best Current Practice, OpenID Connect, and GDPR principles.

Security domain	Primary risk	Required control
Location and tracking	Unauthorized live tracking or stalking	Explicit sharing consent, scoped share tokens, expiration, audit logs, private-by-default settings
APIs	Broken object-level authorization and scraping	Per-object authorization middleware, tenant scopes, rate limits, anomaly detection
AI briefing	Hallucinated or manipulated flight advice	Grounded context only, no-data refusal, safety blocker precedence, prompt injection filtering
Weather pipeline	Tampered forecast input or stale data	Source integrity checks, run freshness, checksum, model-run provenance, validation gates
Payments	Entitlement fraud	Webhook signature verification, idempotent billing events, server-side entitlement checks

2. Scope, Assumptions, and Trust Boundaries

The security architecture covers web, mobile, backend APIs, weather ingestion, AI/ML systems, storage, infrastructure, billing, and organizational operations. It assumes the platform will first launch with consumer pilots and then expand to clubs, schools, events, rescue workflows, and API customers.

The platform must be secure under multi-tenant conditions. A club admin must never see another club's private pilot data. A school instructor must only access assigned students. A rescue operator must access emergency data only through audited emergency workflows.

Boundary	Trusted side	Untrusted side	Controls
Public API Gateway	Backend services	Internet/mobile clients	WAF, TLS, OAuth2/OIDC, rate limits, request validation
Weather Ingestion	Canonical forecast store	External providers and station devices	Provider allowlist, checksums, freshness checks, station signing
AI Context Builder	Grounded structured data	User prompt and public comments	Prompt injection filtering, tool allowlist, no free weather facts
Tracking Service	Authorized telemetry consumers	Anonymous viewers/share links	Scoped tokens, expiry, geofencing, privacy redaction
Admin Console	Ops/security staff	Support users	MFA, RBAC, audit trails, break-glass controls

3. Security Principles and Control Objectives

The platform security model uses explicit controls and measurable objectives. Each security requirement must be testable and traceable to an owner service or operational runbook.

- Privacy by default: exact live location is private unless the user explicitly shares it.
- Least privilege: every token, service account, admin action, and API key receives the minimum permissions needed.
- Object-level authorization: every resource read/write checks ownership, tenant, role, and purpose.

- Safety blocker precedence: AI or scoring systems cannot override hard safety blockers such as stale weather, storm risk, or missing data.
- Verifiable provenance: every derived forecast, alert, or AI briefing links to model run, timestamp, source, and confidence.
- Fail closed for safety decisions: if data is missing, stale, contradictory, or corrupted, the platform returns NO DATA or NO-GO rather than optimistic advice.
- Human accountability: admin, rescue, billing, and moderation actions are audited with actor, reason, timestamp, and target object.

SECURITY_OBJECTIVE_TEMPLATE:

```
id: SEC-API-001
title: Object-level authorization on launch, track, and user resources
owner: api-gateway-service + resource service owner
test: attempt horizontal access with valid token and foreign object id
expected: 403 with audit event authz_denied
mapped_controls: OWASP API1:2023, ASVS access control, GDPR confidentiality
```

4. Asset Inventory and Data Classification

Security implementation starts with data classification. Weather grid data is normally public/low sensitivity. Exact user location, live tracks, rescue requests, payments, and identity data are high sensitivity. AI prompt logs and pilot notes can contain personal data and must be minimized.

Asset	Examples	Classification	Retention baseline	Protection
Identity data	Email, account id, login provider, MFA state	Confidential	Account lifetime + legal retention	Encryption at rest, restricted support access
Precise location	Live GPS, trackpoints, last known position	Highly confidential	User configurable; default limited retention	Private by default, encryption, sharing tokens, access logs
Flight tracks	IGC/GPX, altitude, speed, route	Confidential unless published	User controlled	Visibility settings, anonymized public statistics
Weather data	Forecast cubes, grid tiles, radar, model runs	Public/internal	Operational retention by source/license	Integrity and freshness controls
AI context and output	Briefings, prompts, retrieved forecast context	Confidential	Short retention unless user saves briefing	Prompt logging minimization, redaction
Payment data	Subscription state, invoice IDs, provider customer ID	Confidential/regulated	Legal/business retention	No card storage, webhook verification
Club/school records	Student progress, member rosters, incidents	Confidential	Tenant policy controlled	Tenant isolation, admin audit
Rescue workflow data	Emergency contact, last-known track, search notes	Highly confidential	Strict emergency retention	Break-glass, strong audit, role separation

5. Threat Model and Abuse Cases

The platform threat model prioritizes privacy, safety, and abuse resistance. The main attacker classes are: anonymous internet attackers, authenticated malicious users, jealous/stalking actors, competing scrapers, compromised station devices, malicious club admins, prompt-injection attackers, supply-chain compromise, and operational mistakes.

Threat	Impact	Likelihood	Mitigations
BOLA on /tracks/{id}	Exposure of private flight tracks	High	Resource ownership checks, RLS, authz test suite, audit event
Unauthorized live tracking	Physical safety/privacy risk	High	Private by default, share token expiry, viewer limits, safety UI
Prompt injection in AI briefing	Unsafe advice or data disclosure	Medium	Context isolation, instruction hierarchy, no tool execution from user text
Weather data tampering	Wrong GO/NO-GO recommendation	Medium	Checksums, source signatures where possible, model disagreement gates

Station device spoofing	False reality layer	Medium	Device certificates, station health, anomaly detection, reputation
Entitlement bypass	Revenue loss	High	Server-side entitlement checks, signed webhooks, idempotency
Admin misuse	Tenant data exposure	Medium	RBAC, just-in-time admin access, audit, approval for sensitive exports
Scraping forecast/API	Costs and data licensing issues	High	Rate limits, API keys, bot defense, tiered quotas

6. Identity Architecture

Identity is based on external OIDC/OAuth2 login for consumer users plus optional enterprise SSO for schools/clubs/events. The platform issues short-lived access tokens and rotating refresh tokens. Sensitive operations require step-up authentication.

Principal type	Authentication method	Token scope	Notes
Consumer pilot	Email magic link, passwordless, social OIDC, optional MFA	user:* limited to owned resources	Exact location private by default
Club admin	OIDC + MFA required	club:{id}:admin	May manage club launches and rosters
School instructor	OIDC + MFA required	school:{id}:instructor	Access assigned students only
Event operator	OIDC + MFA required	event:{id}:operator	Access event pilots/tasks
Rescue operator	Enterprise identity + MFA + approval	rescue:case:{id}	Break-glass workflow only
Station device	mTLS or signed device token	station:{id}:write	No user data access
Internal service	Workload identity	service-specific	No shared static credentials

```
JWT_CLAIMS_EXAMPLE:
{
  "sub": "usr_01HV...",
  "iss": "https://auth.paragliding-intel.example",
  "aud": "paragliding-api",
  "exp": 1770000000,
  "scope": ["launch:read", "track:write", "ai:briefing"],
  "tenant_roles": {
    "club_123": ["club_member"],
    "school_88": ["student"]
  },
  "mfa": true,
  "risk_level": "normal"
}
```

7. Authentication Flows

The production system should implement Authorization Code with PKCE for mobile and web clients. The implicit flow must not be used. Refresh tokens are rotated and reuse detection invalidates the session family. High-risk actions require recent authentication or MFA step-up.

Flow	Used for	Security requirements
Authorization Code + PKCE	Mobile and web login	PKCE required, secure redirect URIs, state/nonce validation
Refresh token rotation	Long-lived mobile session	Reuse detection, device binding where possible, revoke on anomaly
Step-up authentication	Export tracks, rescue access, billing changes, admin role grants	Recent login or MFA challenge
Device identity	Weather stations	mTLS certificate or signed JWT; rotate credentials
Service identity	Microservice-to-microservice	Workload identity, mTLS/service mesh optional

```

AUTHENTICATION_POLICY:
  access_token_ttl: 15m
  refresh_token_ttl: 30d
  refresh_rotation: true
  mfa_required_for_roles: [club_admin, school_admin, event_operator, rescue_operator, platform_admin]
  blocked_flows: [oauth_implicit, password_grant]
  mobile_storage: platform_secure_storage_only

```

8. Authorization: RBAC, ABAC, and Row-Level Security

Authorization is the highest-risk area because many objects are user-owned, tenant-owned, share-token-visible, or emergency-accessible. Each service must use a shared authorization library plus database-level safety for sensitive tables.

Layer	Responsibility	Example
API gateway	Authenticate token, coarse route scope, rate limit	Token has track:read
Resource service	Object-level policy	User owns track or has active share token
Database	Row-level protection for sensitive data	tenant_id and owner_user_id filters
Audit layer	Record denied and sensitive granted access	authz_denied, rescue_break_glass

```

ABAC_POLICY_TRACK_READ:
allow if resource.visibility == "public"
allow if resource.owner_user_id == principal.sub
allow if active_share_token(resource.id, request.share_token, now())
allow if principal.role == "event_operator" and resource.event_id in principal.allowed_events
allow if principal.role == "rescue_operator" and active_rescue_case(resource.id, principal.sub)
deny otherwise

```

```

-- Example PostgreSQL RLS concept for private tracks
ALTER TABLE flight_tracks ENABLE ROW LEVEL SECURITY;
CREATE POLICY flight_tracks_owner_policy ON flight_tracks
  USING (owner_user_id = current_setting('app.user_id')::uuid);

-- Application sets app.user_id per request transaction after token validation.

```

9. API Security Architecture

API security follows a deny-by-default model. All endpoints define authentication requirement, scopes, tenant context, object-level authorization checks, rate limits, idempotency, validation schemas, and error behavior. Sensitive endpoints must never leak whether a private object exists to unauthorized users.

Control	Implementation rule
Input validation	JSON schema/Pydantic validation on all requests; reject unknown dangerous fields
Object authorization	Every endpoint with path id must call authz.check(resource, action, principal)
Rate limiting	Different quotas for anonymous, free, pro, school, API customer, and device tokens
Idempotency	Required for billing, alert creation, track upload chunks, station observation batches
Pagination	Cursor-based pagination for list endpoints; max page size enforced
Error model	No stack traces; use stable error codes; do not reveal private object existence
Anti-automation	Bot detection and quota controls for scraping-heavy endpoints

```

STANDARD_ERROR_RESPONSE:
{
  "error": {
    "code": "AUTHZ_DENIED",
    "message": "You are not allowed to access this resource.",

```

```

        "request_id": "req_01HX...",
        "support_ref": "sec-authz-403"
    }
}

FASTAPI_AUTHZ_PATTERN:
@router.get("/tracks/{track_id}")
async def get_track(track_id: UUID, principal=Depends(authn)):
    track = await track_repo.get_metadata(track_id)
    await authz.require(principal, "track.read", track)
    audit.sensitive_read(principal, "track", track_id)
    return await track_service.render_safe(track, principal)

```

10. Mobile Application Security

The mobile app is safety-adjacent and location-heavy. It must comply with mobile security verification practices: secure storage, certificate validation, minimal permissions, controlled background location, safe deep links, encrypted local cache where needed, jailbreak/root risk handling for highly sensitive functions, and explicit user consent.

Mobile area	Requirement
Token storage	Use iOS Keychain / Android Keystore via secure storage; never AsyncStorage/shared preferences
Location permission	Request only when feature needs it; explain background tracking clearly
Offline cache	Encrypt sensitive cached tracks and user profile data
Deep links	Validate host/path; never perform state-changing actions directly from link
Network	TLS required; optional certificate pinning for auth/API domains after operational risk review
Screenshots	Block screenshots on rescue/admin screens where OS allows
Telemetry	Do not log exact GPS coordinates into analytics events by default

```

MOBILE_PERMISSION_COPY_REQUIREMENT:
Feature: Live Tracking
Prompt: "Share live position during this flight so chosen viewers can follow you. You can stop sharing at any time."
Default: off
Background: only while active flight tracking is enabled
Post-flight: prompt user whether to save, publish, anonymize, or delete track

```

11. Location Privacy and Live Tracking Controls

Location privacy is the most sensitive product area. The app must avoid normalizing public exact tracking. Sharing should be temporary, intentional, visible, revocable, and auditable.

Feature	Privacy control
Live tracking	Off by default; explicit start; visible active indicator; one-tap stop
Share links	Random high-entropy tokens, expiration, optional passcode, viewer cap
Public tracks	User must publish intentionally; offer start/end redaction and home-zone privacy
Club visibility	Club membership does not imply exact live tracking access
School visibility	Student/instructor visibility limited to training sessions and policy
Rescue access	Break-glass only with case id, reason, duration, and audit
Analytics	Aggregate geographic heatmaps must be anonymized and thresholded

```

SHARE_TOKEN_MODEL:
share_id: shr...
track_id: trk...

```

```

scope: live_view | replay_view | emergency_view
expires_at: timestamp
viewer_limit: 20
created_by_user_id: uuid
revoked_at: nullable timestamp
audit_required: true

```

12. Data Protection, Encryption, and Key Management

Encryption is required both as a security control and as a privacy assurance. Encryption alone is not sufficient; access control, logging, retention, and deletion must also be implemented.

Data state	Control
In transit	TLS 1.2+ minimum; prefer TLS 1.3; HSTS for web
At rest - database	Managed disk encryption plus column encryption for highly sensitive fields where appropriate
At rest - objects	Server-side encryption for IGC/GPX files, AI context archives, backups
Secrets	Stored in secret manager or sealed secrets; never committed to Git
Keys	KMS-backed rotation; separate prod/staging keys; emergency revocation runbook
Backups	Encrypted, access-restricted, restore-tested

KEY_SEPARATION:

```

k_app_general: general app data encryption
k_location_sensitive: precise live/track encryption
k_backup_prod: production backup encryption
k_webhook_signing: outbound webhooks
k_station_device_ca: weather station device certificates
rotation: 90d for secrets, 180-365d for selected encryption keys depending on service impact

```

13. GDPR and Privacy Operations

For EU users, the product must support privacy rights and data governance from the start. The platform determines why and how most user data is processed, so it will normally act as data controller for consumer users and may act as processor for some school/club enterprise arrangements depending on contracts.

Privacy operation	Implementation requirement
Consent	Separate consent for marketing, live sharing, public track publication, and optional analytics
Data access request	Export profile, tracks, observations, subscriptions, and saved AI briefings
Deletion request	Delete or anonymize user-owned data subject to legal and safety retention limits
Data minimization	Do not store raw AI prompts longer than required unless user saves briefing
Purpose limitation	Do not reuse precise location for advertising or unrelated profiling
DPIA	Perform before launch due to precise location and safety-related processing
Processor contracts	Required with cloud, email, push, analytics, and payment providers

PRIVACY_EXPORT_BUNDLE:

```

/user_profile.json
/pilot_profile.json
/tracks/*.igc
/tracks/metadata.json
/alerts.json
/ai_briefings.json
/billing_portal_link.txt
/consents.json

```


14. AI and LLM Security

The AI briefing system must be treated as a constrained explanation layer, not an autonomous weather authority. It must only summarize structured, retrieved, trusted platform data. It must refuse when data is stale, missing, or blocked by safety rules.

LLM risk	Control
Prompt injection	Separate user text from trusted context; never allow user content to override system safety rules
Insecure output	Validate output schema; post-process for safety-blocker consistency
Sensitive information disclosure	Do not include private tracks or account data unless user is authorized
Model DoS/cost abuse	Token budget, rate limits, tier quotas, caching
Excessive agency	No autonomous execution of billing, sharing, or emergency actions
Hallucinated weather	No free generation of forecast facts; every answer must reference backend data ids

```
AI_BRIEFING_GUARDRAIL:
if forecast.freshness_status != "fresh": return NO_DATA
if safety_blockers.any(): return NO_GO with blockers
if model_disagreement == "high": return MAYBE with uncertainty
llm_context = build_from_structured_records_only(site, forecast, risks, pilot_profile)
llm_output = generate_explanation(llm_context)
validated = validate_json_schema(llm_output)
assert validated.status in allowed_status_from_rules
```

15. ML, Weather Data, and Model Supply Chain Security

The weather and ML systems can influence flight decisions. Source integrity, model provenance, and data leakage prevention are safety and security requirements.

Component	Security requirement
Forecast provider ingestion	Provider allowlist, run timestamps, checksum where available, freshness status
Station data	Device identity, anomaly detection, impossible value rejection, health metrics
Pilot reports	Reputation, abuse detection, moderation, no single untrusted report can override safety
Feature store	Point-in-time features, anti-leakage validation, access controls
Model registry	Signed artifacts, reproducible training metadata, approval gates
Inference	Versioned model ids in outputs, rollback mechanism, shadow testing

```
MODEL_PROMOTION_GATE:
requires:
  - offline_metric_improvement >= configured_threshold
  - no_known_leakage_features: true
  - site_level_error_not_worse_than_baseline: true
  - safety_review_signed: true
  - rollback_plan_exists: true
production_rollout:
  stage_1_shadow: 7d
  stage_2_10_percent: 3d
  stage_3_full: manual_approval
```

16. Payment and Subscription Security

Paid-tier logic must be server-side. The client may display a tier, but the backend must enforce every premium feature through an entitlement service. Payment provider events must be verified and idempotent.

Billing control	Implementation
-----------------	----------------

No card storage	Use payment provider hosted checkout/customer portal
Webhook verification	Verify provider signature and timestamp before processing
Idempotency	Store event id; ignore duplicates; retry safely
Entitlement source of truth	Server-side entitlement table updated from verified payment events
Quota enforcement	API gateway checks tier and usage before expensive calls
Refund/cancel	Grace period and entitlement expiry policy explicit

```

ENTITLEMENT_CHECK:
feature = "ai_xc_planner"
principal = request.principal
entitlement = entitlement_service.get(principal.user_id)
if not entitlement.allows(feature):
    raise PaymentRequired(feature=feature, upgrade_url="/billing/upgrade")

```

17. Cloud, Kubernetes, and Network Security

Infrastructure security inherits from Volume 08, but this volume defines security-specific baseline controls. Production clusters must isolate public ingress, internal services, data stores, and admin tools. Network policies restrict east-west traffic.

Layer	Control
Ingress	TLS termination, WAF/rate limit, request size limits, bot controls
Service mesh/network	Deny-by-default network policies for namespaces
Kubernetes RBAC	Least privilege service accounts; no default admin tokens
Containers	Non-root, read-only FS where possible, image scanning, pinned base images
Databases	Private network only, no public exposure, restricted security groups
Admin access	VPN or zero-trust access, MFA, audited sessions

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-api-to-weather-service
  namespace: prod
spec:
  podSelector:
    matchLabels:
      app: weather-service
  policyTypes: [Ingress]
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: api-gateway
      ports:
        - protocol: TCP
          port: 8080

```

18. Secrets Management

Secrets are operationally dangerous because the platform integrates forecast APIs, payment providers, notification providers, auth providers, and station devices. The design must prevent secret sprawl.

- No secrets in Git, container images, mobile binaries, screenshots, ticket comments, or logs.
- Separate secrets by environment: dev, staging, production.
- Use short-lived workload identity where possible instead of static keys.
- Rotate provider keys after employee offboarding, suspected leak, or periodic rotation interval.
- Support emergency revocation of station device certificates and API customer keys.

SECRET_INVENTORY_FIELDS:

secret_id, provider, environment, owner_service, created_at, rotated_at,
rotation_period_days, stored_in, used_by, emergency_revocation_steps, last_accessed_at

19. Logging, Audit, and Security Monitoring

The platform needs both operational logs and security audit events. Audit logs must be structured, immutable enough for investigation, and protected from normal application users. Logs must avoid sensitive content such as exact GPS coordinates unless explicitly needed and protected.

Event type	When emitted	Required fields
auth.login_success	User logs in	user_id, method, device, ip_hash, risk
authz.denied	Authorization fails	principal, action, resource_type, resource_id_hash
track.share_created	User creates live share	track_id, expiry, scope, actor
rescue.break_glass	Emergency access is used	case_id, actor, reason, expiry, approver
ai.no_data_refusal	AI refuses due to missing/stale data	site_id, reason, model_run_id
billing.webhook_verified	Payment event processed	provider_event_id, subscription_id
station.anomaly	Station data rejected	station_id, variable, value, reason

SECURITY_ALERT_RULES:

- More than 10 AUTHZ_DENIED events for different object ids by same user in 5 min -> investigate BOLA probing
- Share link views from >5 countries in 1 hour -> revoke or challenge
- Rescue break-glass outside approved case -> page security lead
- Billing webhook signature failures >3 in 10 min -> investigate spoofing
- AI prompt budget abuse over tier quota -> throttle and flag account

20. Incident Response and Business Continuity

Incident response must be planned before launch. Location and rescue data incidents have higher urgency than generic availability issues because they can affect personal safety.

Severity	Examples	Response target	Actions
SEV-1	Live location leak, rescue data exposure, active system compromise	Immediate page	Contain, revoke, preserve evidence, notify leadership/legal
SEV-2	Billing webhook abuse, AI unsafe recommendation bug, forecast stale displayed as fresh	1 hour	Disable feature/roll back, notify affected users if needed
SEV-3	Non-sensitive outage, degraded map tiles, delayed alerts	Business hours or on-call	Fix, postmortem if user impact

26. Triage: classify data type, scope, active exploitation, and safety impact.
27. Contain: revoke keys/tokens, disable affected endpoint, block abusive account/IP, freeze suspect station data.
28. Eradicate: patch code, rotate secrets, repair config, remove malicious data.
29. Recover: restore safe service, run verification tests, monitor for recurrence.
30. Notify: follow legal/GDPR and contractual obligations.
31. Postmortem: document root cause, timeline, impact, prevention tasks, owner, due date.

21. Security Testing Program

Security testing must be continuous. Manual pentesting alone is not enough. The platform needs automated tests for authorization, API abuse, mobile storage, AI guardrails, and infrastructure misconfiguration.

Test type	Frequency	Owner	Scope
SAST	Every pull request	Engineering	Backend, mobile, infra code

Dependency scanning	Every pull request and nightly	DevOps	Python, Node, Flutter, container images
DAST/API fuzzing	Nightly staging	Security/QA	API gateway, auth flows, upload endpoints
BOLA regression tests	Every PR touching APIs	Backend	Foreign object id access attempts
Mobile security review	Each release	Mobile lead	Secure storage, permissions, deep links
LLM red-team tests	Each AI prompt/model release	AI lead	Prompt injection, data leakage, unsafe advice
Penetration test	Before paid launch and annually	External tester	Full system

BOLA_TEST_CASE:
 Given user_A owns track_A and user_B owns track_B
 When user_A requests GET /tracks/{track_B}
 Then API returns 403 or 404 privacy-preserving response
 And audit event authz.denied is emitted
 And no track_B content appears in response, logs, or cache

22. Compliance Mapping

This section maps platform controls to widely used frameworks. It is not a certification claim. It is an engineering traceability matrix for implementation and future audits.

Framework/reference	Used for	Platform implementation areas
OWASP ASVS 5.0	Application security requirements baseline	Auth, sessions, validation, access control, logging, cryptography
OWASP API Security Top 10 2023	API threat model	BOLA, auth, resource consumption, SSRF, inventory
OWASP MASVS	Mobile security baseline	Secure storage, networking, platform interaction, privacy
OWASP LLM Top 10 2025	AI app threat model	Prompt injection, output handling, model DoS, supply chain
NIST CSF 2.0	Security governance taxonomy	Identify, Protect, Detect, Respond, Recover, Govern
OAuth 2.0 BCP / RFC 9700	Modern OAuth implementation guidance	PKCE, refresh rotation, avoiding insecure flows
OpenID Connect Core	Authentication and identity claims	OIDC login, claims, nonce/state validation
GDPR	EU privacy obligations	Lawful basis, rights, minimization, processor/controller mapping

23. MVP Security Slice

The MVP cannot implement every enterprise control, but it must include the controls that protect users from the highest-risk failures: account compromise, private location exposure, authorization bugs, stale data, payment abuse, and AI hallucination.

MVP requirement	Acceptance criteria
OIDC/OAuth2 login with PKCE	Mobile and web clients use code flow with PKCE; implicit flow disabled
Object-level authorization library	All user-owned and tenant-owned endpoints call shared authz check
Private-by-default tracking	New tracks and live sessions are private unless explicitly shared
Signed/expiring share links	Share tokens expire and can be revoked
Server-side entitlements	Premium APIs reject free users even if client UI is modified
AI no-data and safety-blocker rules	AI cannot produce GO when stale data or hard blocker exists
Structured audit events	Auth, authz, share, billing, AI safety, admin events emitted
Basic security monitoring	Authz probing, webhook failures, station anomaly, AI abuse alerts configured
Encrypted backups	Nightly backup and restore test procedure exists

24. Security Backlog and Acceptance Criteria

Security work should be managed as product backlog items with owners and release gates. The following backlog is ordered for the next stages of the platform.

Priority	Backlog item	Done when
P0	Implement authorization middleware and BOLA regression tests	100% protected endpoints have tests; foreign object access denied
P0	Implement live-track privacy model	Share tokens expire/revoke; private default verified on mobile
P0	Implement payment webhook verification	Invalid signatures rejected; duplicate events ignored
P0	Implement AI briefing safety validator	Unsafe/no-data test cases produce refusal or NO-GO
P1	Enable database RLS for sensitive tables	Tracks and rescue data protected by DB-level policies
P1	Add admin just-in-time access	Support cannot view sensitive data without reason and audit
P1	Station device certificate system	Compromised station can be revoked without global outage
P2	External penetration test	Critical/high findings fixed before public paid launch
P2	Privacy dashboard	Users can export/delete data and manage consents

25. References

The document is designed against the following public standards and implementation references. Exact framework requirements should be reviewed again before certification, procurement, or formal legal/compliance use.

- OWASP Application Security Verification Standard (ASVS) 5.0 - application security verification requirements.
- OWASP API Security Top 10 2023 - API-specific risk categories including broken object-level authorization and unrestricted resource consumption.
- OWASP Mobile Application Security Verification Standard (MASVS) - mobile app security and privacy baseline.
- OWASP Top 10 for Large Language Model Applications 2025 - LLM-specific risks such as prompt injection, insecure output handling, and model denial of service.
- NIST Cybersecurity Framework 2.0 - governance and cybersecurity risk management outcomes.
- RFC 9700 OAuth 2.0 Security Best Current Practice - modern OAuth security guidance.
- OpenID Connect Core 1.0 - authentication built on OAuth 2.0 and claims-based identity.
- European Commission GDPR/data protection guidance - controller/processor obligations and privacy rights.

Appendix A - Launch Readiness Security Checklist

Gate	Checklist item
Auth	All production roles require MFA except basic consumer pilot accounts; admin roles always MFA
Authz	Every endpoint with object id has explicit policy and regression test
Privacy	Live tracking off by default and share links expire
AI	Safety blocker validator is outside the LLM and cannot be overridden by prompt
Weather	Forecast freshness displayed and stale data cannot produce GO
Billing	Entitlements are enforced server-side and webhook signatures verified
Infrastructure	Databases are private-network only and backups are encrypted
Monitoring	Security alerts configured and tested
Incident Response	SEV-1 runbook exists and on-call contacts are defined